Quantum Computing Project Report

California Housing: Classical and Quantum Regression

A parameterized quantum circuit extension of
Revisiting Deep Learning Models for Tabular Data

Project focus	Continuous regression on tabular data
Quantum model	Eight-qubit hybrid quantum neural network
Execution	Ideal CPU simulation using PennyLane and PyTorch
Experiment	Shared data split; classical baselines; saved checkpoints
Report date	5 November 2026

Report overview

Abstract
1. Introduction & Problem Statement
2. Hardware & Environment Specifications
3. Methodology & Implementation Pipeline
4. Performance & Comparative Analysis
5. Conclusion & Future Scope
References

This report describes the completed local experiment. The assigned paper is classical; the quantum model is this project's extension. The results do not establish quantum advantage.

Abstract

This project investigates continuous house-value prediction using classical machine learning and a hybrid quantum neural network (QNN). It uses the StatLib California Housing dataset, containing 20,640 census block-group records, and draws inspiration from the tabular ResNet architecture in *Revisiting Deep Learning Models for Tabular Data* [1]. The quantum circuit is an additional project implementation, rather than a model proposed in the paper.

Eight numerical features are derived from the supplied raw data. To fit a modest CPU budget, the experiment uses 2,048 training records, 512 validation records, and 4,128 held-out test records. All models use the same selected examples. The comparison includes a mean predictor, histogram gradient-boosted trees, a compact tabular ResNet, and an eight-qubit QNN. The QNN encodes standardized inputs as rotation angles, applies trainable RY and RX gates with a CNOT ring, and converts eight Pauli-Z expectation values into a continuous prediction using a classical linear readout.

The models run locally without quantum hardware. On the fixed test set, boosted trees achieve the lowest RMSE: $52,278 in historical dollars, followed by the ResNet at $58,011 and the two-layer QNN at $88,468. The QNN improves RMSE by 22.8% relative to the mean predictor, but is less accurate and slower than both trained classical baselines. The result demonstrates an operational, trainable quantum regression pipeline and the importance of measuring performance instead of assuming that quantum models are superior.

Keywords: California Housing; tabular regression; parameterized quantum circuit; hybrid QNN; PennyLane; classical comparison.

How to read the results

RMSE and MAE measure prediction error; smaller values are better. R-squared (R2) measures improvement relative to a constant prediction based on the test-target mean; larger values are better. These are regression metrics, not classification accuracy percentages. Reported dollar values refer to the historical dataset and to area-level median values.

1. Introduction & Problem Statement

1.1 Background

Tabular data consists of rows of examples and columns of features. Here, each row describes a California census block group, and the target is its median housing value. The task is supervised regression: learn a mapping from eight numerical inputs to one continuous output. It is not a prediction of a current individual property price.

The assigned paper evaluates deep learning approaches for tabular data, including ResNet and FT-Transformer [1]. This project implements a small ResNet inspired by that work and introduces a parameterized quantum circuit (PQC) as an alternative learned representation. The full paper benchmark, FT-Transformer, extensive tuning and ensembles are outside the implemented scope.

1.2 Motivation

A PQC contains quantum gates whose angles can be adjusted during training. A classical optimizer updates those angles to reduce prediction error. This makes it possible to study quantum machine learning on an ordinary laptop using a simulator. A fair baseline comparison is needed to determine whether this extra complexity helps on the chosen task.

Problem statement: How well does a shallow, eight-qubit hybrid regressor predict California Housing values compared with classical models under a fixed, low-compute experimental setup?

1.3 Objectives

1. Load and validate the supplied dataset and derive meaningful numerical features.
2. Implement classical baselines and a trainable quantum regression model.
3. Prevent data leakage using separate training, validation and test sets.
4. Compare error, training time and circuit resources using saved results.
5. Provide reproducible code, gate diagrams and clear explanations for demonstration.

Dataset and feature meaning

Input	Meaning / conversion
MedInc; HouseAge	Median income in dataset units; median house age
AveRooms; AveBedrms	Total rooms / households; total bedrooms / households
Population; AveOccup	Block-group population; population / households
Latitude; Longitude	Geographical location
Target	Median house value divided by 100,000

The local audit found no nonfinite values and no duplicate raw rows. Recorded targets range from $14,999 to $500,001; 965 rows (4.68%) occur at the maximum value. This ceiling limits what the target can express.

2. Hardware & Environment Specifications

2.1 Hardware Specifications

Item	Project environment
Laptop specification	Intel Core i5 and 8 GB RAM, as reported by the user
Recorded processor identifier	Intel64 Family 6 Model 140 Stepping 1, GenuineIntel
Operating system	Windows 11; recorded platform build 10.0.26200
Compute allocation	CPU execution with two compute threads
GPU / quantum processor	Neither required nor used for the saved experiment
Quantum simulation	Eight qubits: state vector has 2^8 = 256 complex amplitudes

The small circuit and reduced training subset keep this implementation practical on a basic laptop. Training may increase CPU usage. Memory use for the complete process was not benchmarked; framework memory and gradient calculations also contribute, so state-vector size alone is not total RAM use. Times in this report are local measurements, not guarantees for another computer.

2.2 Software & Framework Stack

Software	Recorded version	Role
Python	3.12.14	Main implementation language
PennyLane	0.43.0	Quantum gates, QNode, simulator and diagrams
PyTorch	2.5.1+cpu	Neural layers, gradients and optimization
scikit-learn	1.7.2	Mean and boosted-tree baselines; metrics
NumPy / SciPy	2.3.4 / 1.16.2	Numerical processing
Matplotlib	3.10.7	Saved performance and circuit figures

The simulator is PennyLane **default.qubit** with **shots=None**. It computes exact expectation values without sampling noise. The model uses the PyTorch interface and simulator backpropagation. The virtual environment and pinned dependency files support repeatable execution.

The project is a command-line ML/QML implementation. A frontend is unnecessary for this demonstration because the saved models, metrics and circuit explanation can be inspected directly.

3. Methodology & Implementation Pipeline

3.1 Baseline Algorithm Selection

Model	Configuration and purpose
Mean predictor	Predicts the training-target mean; minimum useful baseline.
Boosted trees	Histogram gradient boosting: 200 iterations, 15 maximum leaf nodes, learning rate 0.08, L2 regularization 1.0.
Tabular ResNet	8 inputs -> width 32 -> two residual blocks -> scalar output; 8,897 trainable parameters.
Main QNN	Eight qubits, two encoding/rotation/entangling layers, linear readout; 41 trainable parameters.
Depth comparison	One-layer QNN with 25 parameters; one exploratory seed.

Each ResNet block applies BatchNorm, Linear(32,64), ReLU, dropout 0.1, Linear(64,32) and dropout 0.1, then adds the block input. The residual connection helps information and gradients pass through the network. A final BatchNorm, ReLU and linear layer produce the output.

Data preparation and split

The supplied **cal_housing.data** has nine raw columns described by **cal_housing.domain**. Household ratios are calculated before modeling. With split seed 42, the initial partitions contain 13,209 training-pool rows, 3,303 validation-pool rows and 4,128 test rows. Fixed selections use 2,048 training and 512 validation rows; 13,952 rows remain unused to reduce compute.

Feature means and standard deviations are fitted only on the selected training rows. Neural inputs become z = (x - training mean) / training standard deviation. Neural targets are also standardized for training, then predictions are converted back before evaluation. Trees and the mean baseline use unscaled features and targets. All models see the same underlying examples.

Pipeline

Local data -> audit and feature conversion -> fixed split -> training-only preprocessing -> model fitting -> best validation checkpoint -> held-out test evaluation -> saved metrics, predictions and diagrams.

This is a comparison of complete implemented pipelines. It is not an exact paper reproduction: the paper uses a different preprocessing and tuning protocol. No test examples are used to fit scaling, update parameters or select the best training epoch.

3. Methodology & Implementation Pipeline

3.2 Quantum Circuit Optimization

All eight qubits start in the zero state. Input feature j becomes angle a_j = 2 arctan(z_j), which keeps the angle between -pi and pi. Each layer applies RY(a_j), trainable RY(theta_lj0), and trainable RX(theta_lj1) on every wire, followed by the ordered CNOT ring 0->1, 1->2, ..., 6->7, 7->0. The main model repeats this sequence twice, re-uploading the same inputs in the second layer.

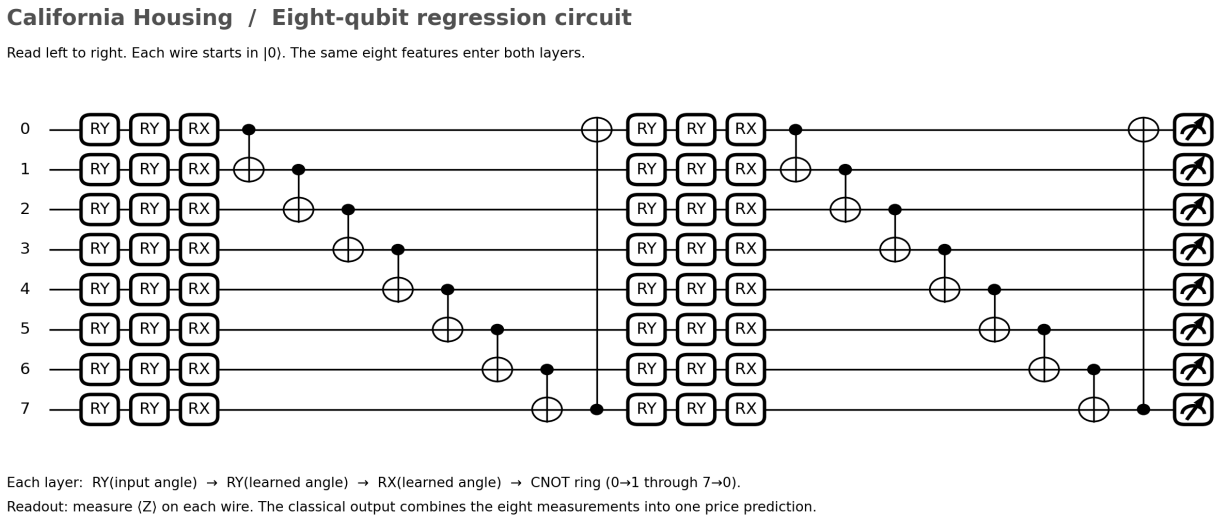


Figure 1. Gate structure drawn from the implemented QNode; angle values are hidden. RY and RX rotate a qubit state. A CNOT uses one control wire and one target wire, allowing correlations between qubits. The drawing itself does not execute training.

The circuit returns eight Pauli-Z expectation values m_j, each between -1 and 1. The classical output is y_scaled = b + sum(w_j m_j). Inverse target scaling gives the reported value. There is no direct classical shortcut from the inputs to the output around the quantum circuit.

Training minimizes mean squared error on standardized targets using AdamW. The QNN learning rate is 0.02; the ResNet learning rate is 0.001. Both use weight decay 0.0001 and gradient-norm clipping at 5. Gradients flow through the simulated circuit and update both circuit angles and linear readout weights.

One epoch means one full pass through the training set. With 2,048 rows and batches of 64, an epoch contains 32 batches. Training allows up to 40 epochs and stops after 10 epochs without a validation-MSE improvement of at least 0.000001. The best validation checkpoint is restored. The main neural models use seeds 7, 17 and 27.

3. Methodology & Implementation Pipeline

3.3 Quantum Resource Optimization

Resource control comes from using eight qubits, shallow circuits, a small training subset, batch processing and two CPU threads. Each feature maps directly to one qubit, so there is no separate dense input network. The two-layer circuit contains 32 learned angles; the output layer adds eight weights and one bias, for 41 trainable parameters.

The one-layer model has 16 learned angles and nine output parameters. It provides an exploratory depth comparison. Fewer parameters do not automatically imply lower runtime: the simulator must propagate quantum states and gradients for every batch.

The adjacent encoding RY(a) and trainable RY(b) gates on the same wire could mathematically merge into RY(a+b). The submitted implementation keeps them separate so that input encoding and learned offsets remain easy to see. No gate-fusion pass, hardware mapping or quantum error correction is claimed.

3.4 Compilation & Execution Engine

The circuit is defined as a PennyLane QNode in **housing/models.py**. PennyLane records the gate operations and executes them on **default.qubit**. PyTorch differentiates the ideal simulator using **diff_method="backprop"**. This is simulator execution, not compilation to a physical quantum device, and it does not use a hardware parameter-shift experiment.

The training loop stores configuration, split indices, preprocessing statistics, model checkpoints, predictions and metrics under **results/main**. Diagram utilities draw the same circuit definition; the saved-model demonstration actually evaluates the model.

How to run the completed project

Run these commands in PowerShell from the project folder with its existing virtual environment. Demonstration and drawing do not retrain the models.

```
.\.venv\Scripts\python.exe run.py demo
```

```
.\.venv\Scripts\python.exe run.py explain
```

```
.\.venv\Scripts\python.exe terminal_circuit.py --ascii --width 58
```

```
.\.venv\Scripts\python.exe generate_diagrams.py
```

For a new training run, choose a new output directory:

```
.\.venv\Scripts\python.exe run.py train --output results/repeat
```

Use **README.md** for installation on another machine. Recreate the virtual environment there rather than copying this laptop's environment.

4. Performance & Comparative Analysis

4.1 Classical vs. Quantum Comparative Matrix

Model	Runs	RMSE ($)	MAE ($)	R2	Train (s)
Mean predictor	1	114,569	91,225	-0.0017	0.001
Boosted trees	1	52,278	35,073	0.7914	0.4
Tabular ResNet	3	58,011	39,563	0.7432	5.9
QNN, two layers	3	88,468	67,103	0.4027	116.9
QNN, one layer*	1	108,619	85,087	0.0997	74.2

Dollar errors refer to historical median values. Repeated-run entries are means. *The one-layer QNN is a single-seed exploratory comparison, not a replacement selected after inspecting test performance.

RMSE = square root of the mean squared prediction error. MAE = mean absolute prediction error. R2 = 1 - (sum of squared errors / sum of squared deviations from the test-target mean). RMSE gives larger errors more weight than MAE. A negative R2 is possible when predictions are worse than that reference constant.

4. Performance & Comparative Analysis

Test-error comparison

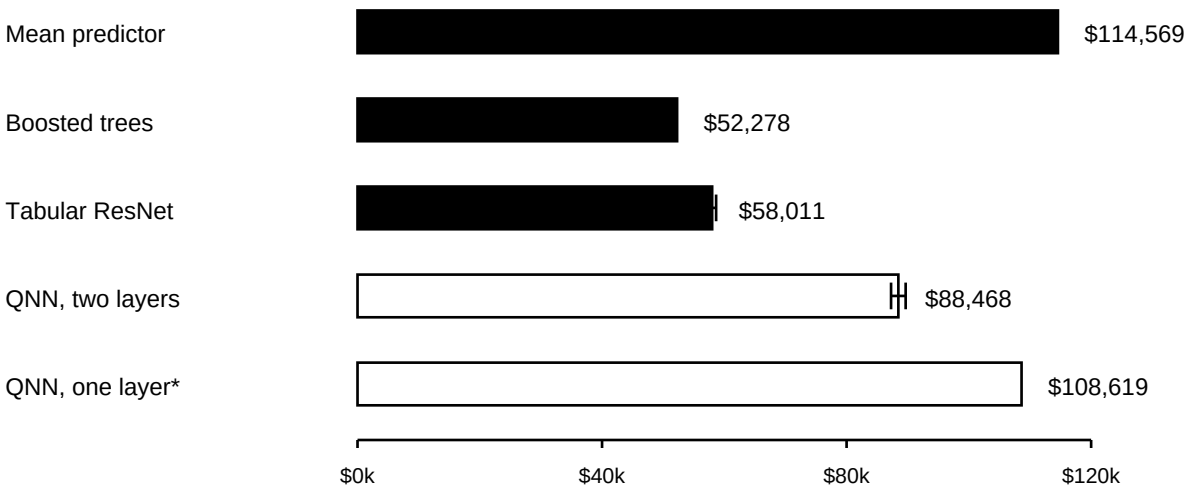


Figure 2. Held-out test RMSE in historical dollars; lower is better. Filled bars: classical models. Outlined bars: QNNs. Whiskers show one sample standard deviation for the three-seed ResNet and two-layer QNN. *The one-layer QNN has one exploratory run.

4.2 Quantum Resource Utilization

Resource	One-layer QNN	Two-layer QNN
Qubits / state amplitudes	8 / 256	8 / 256
Trainable circuit angles	16	32
Classical output parameters	9	9
Total learned parameters	25	41
Readout	8 Z expectations	8 Z expectations
Shots / device	Exact / CPU simulator	Exact / CPU simulator

The expectation values are analytic, not finite-shot measurements. No hardware queue time, device calibration, connectivity overhead or peak-memory measurement was recorded. The resource counts describe the logical circuit actually defined in code.

4. Performance & Comparative Analysis

4.3 Execution Time & Accuracy Analysis

Boosted trees achieve the lowest measured test error and train in about 0.4 seconds. The ResNet averages 5.9 seconds per fit, while the two-layer QNN averages 116.9 seconds. Thus, the QNN takes about 20 times the ResNet training time in this run despite having fewer parameters. Simulator work and differentiation make parameter count alone an unreliable runtime estimate.

Model / seed	Epochs run	Best epoch	Training seconds
Tabular ResNet / 7	40	40	5.61
Tabular ResNet / 17	38	28	5.83
Tabular ResNet / 27	40	37	6.14
QNN, two layers / 7	34	24	111.69
QNN, two layers / 17	32	22	108.04
QNN, two layers / 27	40	39	130.86
QNN, one layer* / 7	40	34	74.17

The mean baseline does not use epochs. Boosted trees use 200 boosting iterations. All nine fits total approximately 7 minutes 23 seconds. These times include training and validation, but exclude dependency installation, final test prediction, plotting and report generation.

Across three seeds, test-RMSE sample standard deviation is approximately $647 for the ResNet and $1,193 for the main QNN. These describe initialization variation on one fixed split, not confidence intervals or variation across different datasets.

4.4 Circuit Depth & Gate Count Analysis

Logical circuit resource	One layer	Two layers
Input RY gates	8	16
Trainable RY gates	8	16
Trainable RX gates	8	16
CNOT gates	8	16
Total gates (excluding readout)	32	64
Logical depth (unmerged gates)	11	22

Each layer has three parallel rounds of single-qubit rotations and an ordered chain of eight dependent CNOT gates. Depth counts the longest gate-dependency path, not simply the number of Python loops. Measurements are excluded. These counts precede any hardware decomposition or gate merging.

4. Performance & Comparative Analysis

4.5 Noise Impact & Error Analysis

No noise experiment was performed. The simulator uses exact expectation values with no finite-shot sampling, gate noise or readout noise. The reported prediction errors therefore measure limitations of the learned regression pipeline, not errors measured on a quantum processor. The experiment cannot establish noise robustness.

Possible contributors to prediction error include the small training subset, fixed model settings, limited circuit expressiveness, and compression of large standardized inputs by the arctangent encoding. These are plausible explanations, not individually established causes. The difference between RMSE and MAE indicates that larger prediction errors affect the aggregate RMSE substantially.

The maximum-value ceiling in the dataset also limits target resolution. Random geographical splits may place nearby block groups in both training and test sets; the measured performance does not establish generalization to entirely new regions or present-day housing prices.

4.6 Experimental Results & Discussion

The two-layer QNN reduces RMSE by 22.8% relative to the mean predictor, showing that this implemented pipeline learns useful predictive structure. However, boosted trees and the ResNet achieve lower error with less training time. The evidence supports quantum regression feasibility on a CPU simulator, not quantum advantage.

The one-layer QNN has RMSE $108,619, compared with $88,468 for the mean of the two-layer QNN runs. This suggests that the selected deeper circuit is more effective in this experiment, but the one-layer comparison uses only one seed. It does not establish a statistically reliable depth effect.

The saved best-checkpoint circuit angles differ from their initial values by L2 distances 5.0030, 4.8408 and 6.6711 for the main QNN seeds. This confirms that the quantum parameters changed during training. Parameter movement alone is not proof of useful learning; held-out comparison supplies the separate performance evidence.

Verification and reproducibility

The saved verification record reports eight passing tests, reproduced checkpoint predictions and reproduced metrics for all nine runs at a comparison tolerance of 1e-10. Checks cover feature conversion, disjoint splits, training-only preprocessing, quantum gradients and updates, and model save/load behavior. Split indices, preprocessing statistics and checkpoints are retained for inspection.

No hyperparameter search or equal-wall-time tuning was performed. Model capacities and preprocessing differ. Conclusions apply to these fixed pipelines and this split, rather than to all classical or quantum models.

5. Conclusion & Future Scope

5.1 Summary of Results

The project successfully implements an eight-qubit, trainable hybrid quantum regressor for California Housing and compares it with classical baselines using shared data. Boosted trees perform best. The QNN beats a constant mean prediction but is slower and less accurate than the trained classical models. The main learning outcome is understanding the complete path from tabular features through quantum encoding, circuit optimization and measurement to a continuous prediction, with evidence-based evaluation.

Short presentation explanation: We extended a classical tabular deep learning study with a simulated quantum neural network. Eight housing features are encoded as rotation angles on eight qubits. Trainable RY/RX rotations and CNOT gates transform the state, and eight Z expectations feed a linear output. We trained and compared the models on the same data. The QNN learned, but boosted trees achieved the best accuracy and runtime in our experiment.

5.2 Future Improvements

Increase the training subset when compute permits; tune models using validation data with a declared budget; repeat the one-layer comparison across multiple seeds; and test several data splits, including geographical holdouts. Compare encodings and entangling layouts through controlled experiments. Add finite-shot and explicit noise simulations before making claims about hardware suitability. Consider hardware execution only after documenting gate mapping and measurement costs.

Further changes should be selected using validation results, with a fresh or strictly protected final test evaluation. Repeatedly redesigning models based on the same test set would weaken the reliability of the reported results.

5.3 GitHub Repository Link

Not provided. The completed source code is currently available in the local QC_QML project folder. Add the team's actual repository URL before submission if a hosted repository is required. The paper authors' repository in the references is not this project's repository.

References

[1] Y. Gorishniy, I. Rubachev, V. Khrulkov and A. Babenko. Revisiting Deep Learning Models for Tabular Data. NeurIPS, 2021. Supplied local PDF: references/2106.11959v5.pdf.

https://arxiv.org/abs/2106.11959

[2] Paper authors' reference implementation. This is the source paper repository, not the student project repository.

https://github.com/yandex-research/tabular-dl-revisiting-models

[3] StatLib California Housing dataset, supplied locally as CaliforniaHousing/cal_housing.data and cal_housing.domain. Dataset archive mirror:

https://figshare.com/articles/dataset/cal_housing_tgz/3829992

[4] Scikit-learn California Housing loader source: feature conversion and target scaling.

https://github.com/scikit-learn/scikit-learn/blob/main/sklearn/datasets/_california_housing.py

[5] PennyLane documentation: PyTorch interface and differentiable quantum models.

https://docs.pennylane.ai/en/stable/introduction/interfaces/torch.html

Local implementation and experiment evidence

housing/models.py: actual ResNet and QNN definitions.
housing/data.py: data conversion and preprocessing.
housing/experiment.py: training and evaluation.
results/main/config.json: environment and experiment settings.
results/main/summary.json: aggregate test metrics.
results/main/training_records.json: epochs, timing and parameter changes.
results/main/verification.json: saved correctness checks.
generate_diagrams.py / terminal_circuit.py: circuit visualization utilities.

Numerical claims in this report come from the completed local run, not from the source paper's benchmark tables. External links identify background sources; no new benchmark or hardware experiment was run while preparing this report.